

LAPORAN AKHIR TEAM BASED PROJECT
E-COMMERCE ORDER MANAGEMENT & RECOMMENDATION
ENGINE

Disusun untuk memenuhi tugas mata kuliah Algoritma dan Struktur Data



Disusun oleh :

Davin junianto (25051030101)

Reino Mifta Suputra (25051030088)

Fransisca Laurency (25051030109)

Fikri Octaviansyah (25051030091)

PROGRAM STUDI S1 TEKNIK ELEKTRO
DEPARTEMEN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA

2026

DAFTAR ISI

DAFTAR ISI	2
BAB I PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Rumusan Masalah	4
1.3 Tujuan Sistem	5
1.4 Batasan Implementasi	5
1.5 Diagram Arsitektur Sistem	6
BAB 2 PERANCANG SISTEM.....	7
2.1 Struktur Data dan Justifikasi	7
2.2 Diagram Interaksi Antar Modul.....	8
2.3 Pseudocode Algoritma Utama	9
BAB III IMPLEMENTASI	15
3.1 Penjelasam Kode Permodul	15
3.2 Screenshot CLI Berjalan	21
3.3 Contoh Input atau Output Nyata	23
BAB IV ANALISIS BIG-O	24
4.1 Analisis Kompleksitas Waktu Dan Ruang Setiap Fungsi Utama	24
4.2 Tabel Ringkasan	25
4.3 Teori Dan Ekperimen	26
BAB V HASIL EKSPERIMEN	27
5.1 Tabel Runtime Eksperimen	27
5.2 Grafik Tren	27
5.3 Diskusi	28
BAB IV JAWABAN PERTANYAAN ANALISIS.....	29
BABVII PENUTUP.....	31
7.1 Kesimpulan	31
7.2 Refleksi Trade-Off Desain	31
7.3 Saran Pengembangan	32
7.4 Lampiran	32

7.4.1 Pembagian Tugas	32
7.4.2 Tabel Runtime	33
7.4.3 Log Ai	33

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi digital telah mengubah cara masyarakat Indonesia dalam berbelanja. Platform e-commerce seperti Tokopedia dan Shopee kini melayani lebih dari 200 juta pengguna aktif dengan jutaan transaksi yang terjadi setiap harinya. Di balik kemudahan berbelanja yang dirasakan pengguna, terdapat sistem pengelolaan order yang sangat kompleks yang bekerja di balik layar untuk memastikan setiap transaksi berjalan dengan cepat, tepat, dan efisien. Namun dalam implementasi sistem e-commerce yang sederhana, terdapat tiga permasalahan mendasar yang sering diabaikan. Pertama, sistem antrian order yang tidak terstruktur menyebabkan pelanggan Premium harus menunggu sama lama dengan pelanggan Economy, padahal pelanggan dengan tingkat prioritas lebih tinggi seharusnya mendapatkan layanan yang lebih cepat. Kedua, pencarian produk dalam katalog yang tidak menggunakan struktur data yang tepat akan menghasilkan waktu pencarian $O(n)$ yang tidak efisien, terutama ketika jumlah produk terus bertambah hingga ribuan bahkan jutaan item. Ketiga, tidak adanya sistem rekomendasi yang berbasis pola pembelian nyata menyebabkan peluang cross-selling hilang begitu saja, padahal data co-purchase antar produk merupakan aset berharga yang dapat meningkatkan nilai transaksi secara signifikan.

Berdasarkan ketiga permasalahan tersebut, proyek ini dirancang untuk mensimulasikan solusi nyata menggunakan pendekatan algoritmik yang tepat. Sistem dibangun menggunakan bahasa pemrograman Python 3 dengan mengimplementasikan seluruh struktur data dari nol tanpa menggunakan library bawaan, meliputi Queue berbasis Linked List untuk manajemen antrian multi-prioritas, Binary Search Tree untuk katalog produk, Graf dengan adjacency list untuk sistem rekomendasi berbasis co-purchase, Stack untuk riwayat transaksi per pelanggan, serta algoritma Bubble Sort dan Insertion Sort untuk pengurutan laporan harian. Seluruh eksperimen dilakukan dengan seed tetap `np.random.seed(99)` untuk menjamin reproducibility hasil pengujian.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam proyek ini adalah sebagai berikut:

1. Bagaimana mengimplementasikan sistem antrian multi-prioritas berbasis Linked List yang dapat mendahulukan pelanggan Premium dibanding Regular dan Economy dengan operasi enqueue dan dequeue $O(1)$?
2. Bagaimana membangun katalog produk menggunakan Binary Search Tree sehingga proses pencarian dan pembaruan stok dapat dilakukan secara efisien dengan kompleksitas $O(\log n)$?
3. Bagaimana merancang sistem rekomendasi produk berbasis graf co-purchase menggunakan algoritma BFS untuk menghasilkan rekomendasi produk yang relevan berdasarkan pola pembelian pelanggan?
4. Bagaimana membandingkan performa Bubble Sort dan Insertion Sort yang diimplementasikan pada Linked List untuk pengurutan laporan harian pada ukuran data $N = 50, 100, \text{ dan } 300$?

1.3 Tujuan Sistem

Berdasarkan rumusan masalah yang telah diuraikan, tujuan dari proyek ini adalah sebagai berikut:

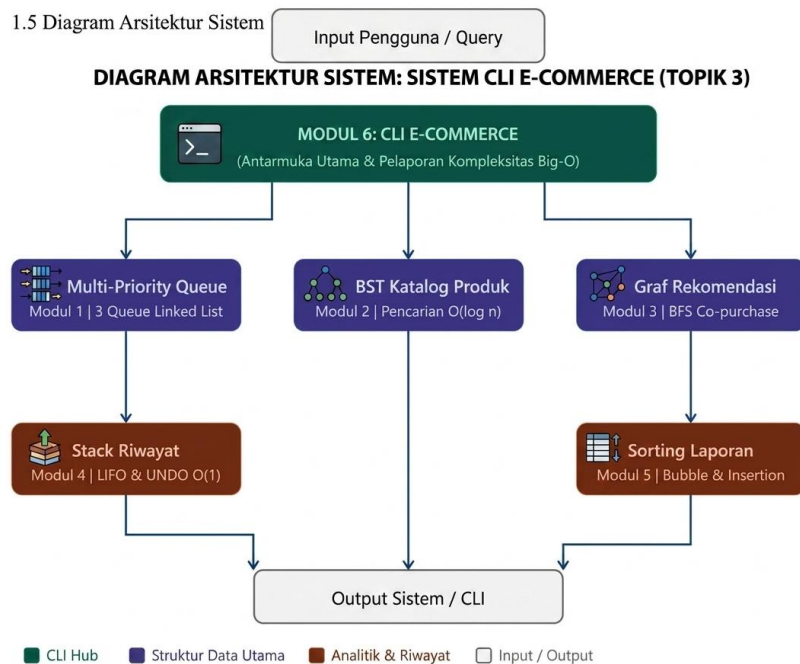
1. Membangun sistem antrian multi-prioritas menggunakan tiga Queue berbasis Linked List untuk tier Premium, Regular, dan Economy sehingga pelanggan dengan prioritas lebih tinggi selalu dilayani terlebih dahulu dengan operasi $O(1)$.
2. Mengimplementasikan Binary Search Tree sebagai katalog produk yang mampu melakukan pencarian, penambahan, dan pembaruan stok 100 produk secara efisien dengan kompleksitas $O(\log n)$ tanpa menggunakan library bawaan Python.
3. Membangun sistem rekomendasi produk berbasis graf co-purchase menggunakan algoritma BFS dengan batas maksimal 2 hop untuk menghasilkan rekomendasi produk yang relevan berdasarkan pola pembelian bersama pelanggan.
4. Membandingkan performa Bubble Sort dan Insertion Sort yang diimplementasikan langsung pada Linked List untuk pengurutan laporan harian dan membuktikan algoritma mana yang lebih efisien melalui eksperimen runtime pada $N = 50, 100$, dan 300 order.

1.4 Batasan Implementasi

Agar pembahasan dalam proyek ini tetap terfokus dan terarah, maka ditetapkan batasan implementasi sebagai berikut:

1. Bahasa pemrograman yang digunakan adalah Python 3 tanpa menggunakan library struktur data bawaan. Linked List, Stack, Queue, BST, dan Graph wajib diimplementasikan dari nol.
2. Jumlah produk yang digunakan dalam sistem adalah 100 produk dengan kode P001 hingga P100, dan jumlah pelanggan yang disimulasikan adalah 50 pelanggan dengan kode C001 hingga C050.
3. Jumlah order yang disimulasikan adalah 300 order acak yang dibangkitkan menggunakan seed tetap `random.seed(99)` untuk menjamin reproducibility hasil eksperimen.
4. Tingkat prioritas pelanggan dibatasi pada tiga tier yaitu PREMIUM, REGULAR, dan ECONOMY, tanpa adanya sub-prioritas di dalam masing-masing tier.
5. Stack riwayat transaksi per pelanggan dibatasi maksimal 10 transaksi terakhir. Jika kapasitas penuh, transaksi paling lama akan otomatis digeser keluar.
6. Sistem rekomendasi berbasis BFS dibatasi maksimal 2 hop dari produk sumber, sehingga hanya produk yang terhubung langsung maupun tidak langsung dalam dua lapis yang akan direkomendasikan.
7. Antarmuka sistem menggunakan CLI (Command Line Interface) berbasis teks di terminal, tanpa menggunakan antarmuka grafis maupun aplikasi web.

1.5 Diagram Arsitektur Sistem



Sistem *E-Commerce Order Management & Recommendation Engine* ini beroperasi secara terpusat melalui antarmuka *Command Line Interface* (CLI) yang bertindak sebagai pengontrol utama. Saat pengguna memasukkan perintah, Modul 6 (CLI) akan menerjemahkan instruksi tersebut dan mendistribusikannya ke tiga modul struktur data utama untuk diproses secara langsung di dalam memori (*in-memory*). Modul 1 menggunakan tiga *Priority Queue* terpisah berbasis *Linked List* untuk mengelola antrian pesanan, sehingga sistem selalu memprioritaskan pelayanan pelanggan tier Premium sebelum beralih ke tier Regular dan Economy. Pemrosesan pesanan ini didukung oleh Modul 2 yang menggunakan *Binary Search Tree* (BST) agar pencarian produk dan pembaruan stok berjalan sangat cepat dengan kompleksitas $O(\log n)$, serta Modul 3 yang memanfaatkan struktur Graf dan algoritma *Breadth-First Search* (BFS) untuk memberikan rekomendasi produk tambahan secara cerdas berdasarkan pola pembelian bersama (*co-purchase*).

Selain pemrosesan pesanan harian, sistem ini juga memiliki kapabilitas untuk melacak riwayat dan menghasilkan laporan analitik. Modul 4 menerapkan struktur data *Stack* dengan prinsip LIFO (*Last In, First Out*) guna menyimpan jejak transaksi masing-masing pelanggan, yang sekaligus memfasilitasi fitur pembatalan pesanan terakhir (UNDO_ORDER) secara instan dengan operasi $O(1)$. Pada akhir operasional, data rekapitulasi pesanan akan diarahkan ke Modul 5 untuk diurutkan menggunakan algoritma *Bubble Sort* (berdasarkan total belanja) dan *Insertion Sort* (berdasarkan waktu pesan) yang diimplementasikan murni pada *Linked - List*. Seluruh hasil eksekusi algoritma dari berbagai modul ini, baik berupa status pesanan, daftar rekomendasi, maupun laporan harian, akan dikembalikan ke lapisan CLI untuk ditampilkan ke layar terminal beserta metrik kompleksitas waktunya (Big-O).

BAB 2

PERANCANG SISTEM

2.1 Struktur Data dan Justifikasi

Sistem ini memodelkan platform *e-commerce* untuk melayani manajemen antrean pesanan multi-prioritas, pencarian katalog produk, sistem rekomendasi berbasis *co-purchase*, dan perekaman riwayat transaksi. Berikut adalah struktur data yang digunakan beserta justifikasinya:

2.1.1 Multi-Priority Queue (Queue berbasis Linked List):

- **Penggunaan:** Mengelola antrean pesanan yang masuk berdasarkan tiga tingkat prioritas pelanggan (*PREMIUM*, *REGULAR*, *ECONOMY*) ke dalam tiga *queue* yang terpisah.
- **Justifikasi:** Implementasi menggunakan *Linked List* dengan *pointer tail* memungkinkan operasi penambahan pesanan (*enqueue*) dan pelayanan pesanan (*dequeue/serve*) dilakukan dalam waktu konstan atau $O(1)$. Pemisahan menjadi tiga antrean menjamin pelanggan *Premium* selalu dilayani terlebih dahulu tanpa perlu melakukan proses pengurutan (*sorting*) ulang pada antrean secara keseluruhan.

2.1.2 Binary Search Tree (BST):

- **Penggunaan:** Menyimpan direktori katalog produk (berisi 100 item) dengan *key* berupa *kode_produk* bertipe *string*, guna memfasilitasi pencarian data, penambahan item, dan pembaruan stok produk.
- **Justifikasi:** Pada kondisi rata-rata, struktur pohon BST memberikan kompleksitas waktu $O(\log n)$ untuk operasi *insert* dan *search*. Hal ini jauh lebih efisien untuk pencarian katalog *e-commerce* dibandingkan menggunakan *array/list* biasa yang memakan waktu linear $O(n)$. BST juga memungkinkan pencetakan daftar seluruh produk secara terurut alfabetis (*fast directory*) menggunakan *inorder traversal*.

2.1.3 Graph dengan Adjacency List (Dict of Linked List):

- **Penggunaan:** Memodelkan relasi *co-purchase* (pembelian produk secara bersamaan oleh pelanggan) untuk membangun mesin rekomendasi (*Recommendation Engine*).
- **Justifikasi:** Penggunaan *adjacency list* sangat menghemat penggunaan memori untuk merepresentasikan koneksi antar produk (graf barang) dengan kompleksitas ruang $O(V+E)$. Algoritma penelusuran *Breadth-First Search* (BFS) kemudian diimplementasikan untuk mengeksplorasi graf secara *level-by-level* dengan batas maksimal 2 *hop* untuk menghasilkan rekomendasi produk yang relevan secara efisien dengan kompleksitas waktu $O(V+E)$.

2.1.4 Stack berbasis Linked List:

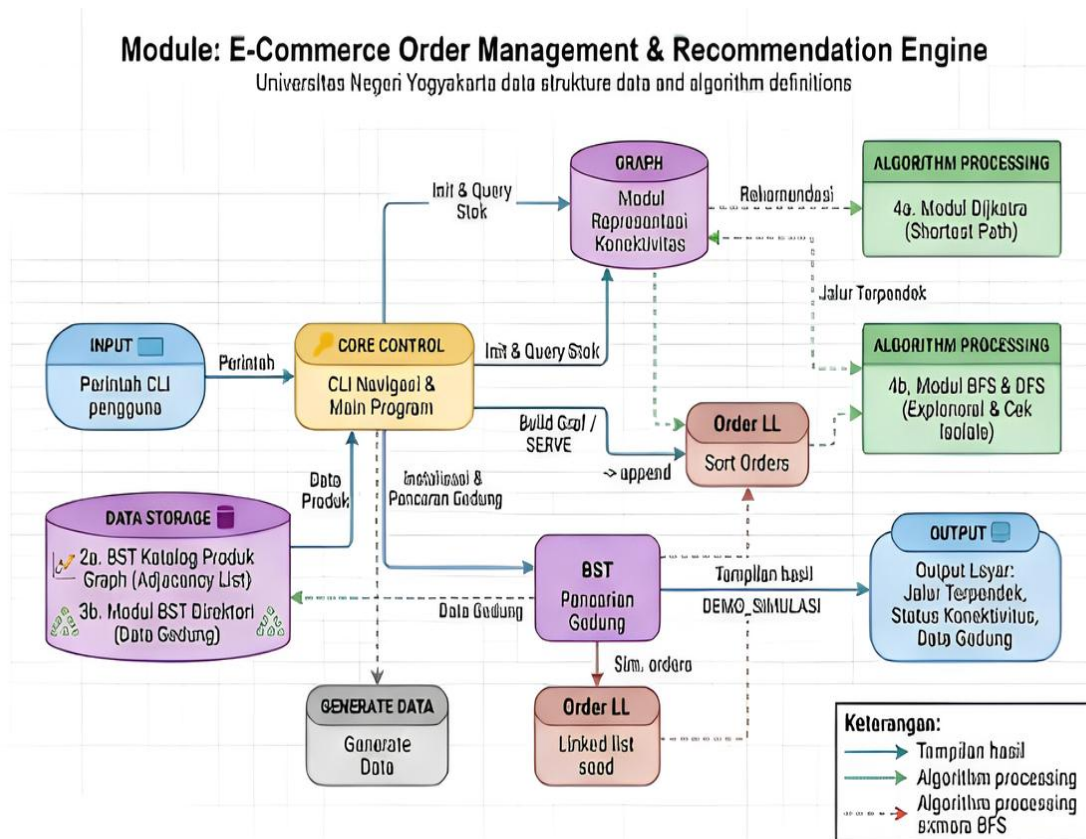
- **Penggunaan:** Menyimpan riwayat transaksi pemesanan independen untuk masing-masing pelanggan (dibatasi maksimal 10 transaksi terakhir) dan menjalankan fungsi pembatalan pesanan.
- **Justifikasi:** Struktur data *Stack* bersifat LIFO (*Last-In, First-Out*), yang merupakan mekanisme paling natural dan mutlak diperlukan untuk fitur *undo* (CANCEL_LAST).

Implementasi dengan *Linked List* menjamin operasi penambahan riwayat (*push*) dan pembatalan transaksi (*pop*) berjalan sangat cepat dalam waktu $O(1)$.

2.1.5 Singly Linked List (untuk Sorting Laporan):

- **Penggunaan:** Menjadi struktur data dasar penampung data rekapitulasi pesanan harian yang kemudian akan diurutkan menggunakan algoritma *Bubble Sort* dan *Insertion Sort*.
- **Justifikasi:** Implementasi pengurutan langsung pada *Linked List* digunakan untuk menganalisis performa algoritma tanpa *library* bawaan. Proses *sorting* ini menguji efisiensi manipulasi *pointer* atau pertukaran *node* (pertukaran data pesanan berdasar harga total atau waktu pemesanan) pada ukuran data yang berbeda, meskipun secara teoritis algoritmanya memakan kompleksitas waktu $O(n^2)$.

2.2 Diagram Interaksi Antar Modul



Penjelasan interaksi :

1. **Pengguna memasukkan perintah melalui CLI** — semua input (ORDER, SERVE, CARI_PRODUK, dst.) diterima modul Core Control (main()).
2. **Core Control menginisialisasi dan mengakses Data Storage** — BST Katalog dimuat dengan `bst.insert()` untuk 100 produk; Queue tier (PREMIUM/REGULAR/ECONOMY) dan Stack (cancel & riwayat pelanggan) dioperasikan saat ORDER dan SERVE dijalankan.

3. **Saat ORDER diproses**, Core Control mencari produk via BST (bst.search()), memasukkan order ke Queue tier yang sesuai, serta mem-push ke cancel_stack.
4. **Saat SERVE diproses**, order di-dequeue dari Queue prioritas, lalu di-append ke Order Linked List dan di-push ke stack riwayat pelanggan (cust_stacks).
5. **DEMO_SIMULASI** menjalankan 300 order acak (seed=99), sekaligus membangun **Graf co-purchase** lewat build_graph() — ini mengisi Graph Rekomendasi (adjacency list).
6. **LAPORAN_HARIAN** mengambil data dari Order Linked List, menjalankan dua modul Algorithm Processing: Bubble Sort (harga DESC) dan Insertion Sort (waktu ASC), lalu hasilnya diteruskan ke Output.
7. **REKOMENDASI** memanggil graf.rekomendasikan() yang menggunakan BFS maksimal 2 hop untuk menemukan produk yang sering dibeli bersamaan — hasilnya tampil sebagai Output rekomendasi.
8. **Semua Output** (laporan, rekomendasi, katalog, detail produk, status antrian) dikembalikan ke CLI sebagai tampilan teks terformat.

2.3 Pseudocode Algoritma Utama

A. Algoritma Queue

Queue diimplementasikan menggunakan Linked List (LLNode) dengan dua pointer head dan tail untuk mendukung operasi enqueue O(1) dan dequeue O(1). Sistem menggunakan tiga antrian terpisah berdasarkan tier pelanggan: PREMIUM, REGULAR, dan ECONOMY.

Kelas LLNode:

Atribut data = NULL
Atribut next = NULL

Kelas Queue (berbasis Linked List):

Atribut head = NULL // pointer depan untuk dequeue
Atribut tail = NULL // pointer belakang untuk enqueue
Atribut _size = 0

Prosedur Enqueue(item):

node_baru = LLNode dengan data = item
Jika tail adalah NULL (antrean kosong):
 head = node_baru
Lainnya:
 tail.next = node_baru
tail = node_baru
_size = _size + 1

Fungsi Dequeue():

Jika head adalah NULL (antrean kosong):
 Kembalikan NULL (atau lempar error)

item_dihapus = head.data
head = head.next

Jika head adalah NULL: // jika antrean menjadi kosong
 tail = NULL

```
_size = _size - 1  
Kembalikan item_dihapus
```

Fungsi IsEmpty():
Kembalikan (_size == 0)

B. Algoritma Stack

Stack diimplementasikan menggunakan Linked List dengan pointer top. Stack digunakan untuk dua keperluan: menyimpan riwayat transaksi per pelanggan (cust_stacks) dan menyimpan order terakhir untuk pembatalan (cancel_stack) dengan kapasitas terbatas.

Kelas Stack (berbasis Linked List):

```
Atribut top = NULL  
Atribut _size = 0  
Atribut kapasitas = 10 // batas maksimum elemen
```

Prosedur Push(item):

```
Jika _size >= kapasitas:  
    items = ambil semua elemen kecuali yang terlama  
    Kosongkan stack (top = NULL, _size = 0)  
    Untuk setiap item dalam reversed(items):  
        Masukkan kembali ke stack  
node_baru = LLNode dengan data = item  
node_baru.next = top  
top = node_baru  
_size = _size + 1
```

Fungsi Pop():

```
Jika top adalah NULL (stack kosong):  
    Kembalikan NULL  
item_dihapus = top.data  
top = top.next  
_size = _size - 1  
Kembalikan item_dihapus
```

Fungsi IsEmpty():

```
Kembalikan (_size == 0)
```

Fungsi ToList():

```
hasil = []  
current = top  
Selama current tidak NULL:  
    Tambahkan current.data ke hasil  
    current = current.next  
Kembalikan hasil
```

C. Algoritma BST Katalog Produk

BST digunakan untuk menyimpan dan mencari katalog produk dengan key berupa kode produk bertipe string (contoh: 'P001', 'P099'). Pencarian membandingkan string secara alfabetis sehingga kompleksitas pencarian rata-rata adalah $O(\log n)$.

```

// Struktur Node BST
Kelas BSTNode:
    Atribut produk // objek Produk (kode, nama, harga, stok)
    Atribut left = NULL
    Atribut right = NULL

Kelas BSTKatalog:
    Atribut root = NULL

Prosedur Insert(produk):
    root = InsertRekursif(root, produk)

Fungsi InsertRekursif(node, produk):
    Jika node adalah NULL:
        Kembalikan BSTNode(produk)
    Jika produk.kode < node.produk.kode:
        node.left = InsertRekursif(node.left, produk)
    Lainnya jika produk.kode > node.produk.kode:
        node.right = InsertRekursif(node.right, produk)
    Kembalikan node

Fungsi Search(kode):
    node = SearchRekursif(root, kode)
    Jika node tidak NULL: Kembalikan node.produk
    Kembalikan NULL

Fungsi SearchRekursif(node, kode):
    Jika node adalah NULL atau node.produk.kode == kode:
        Kembalikan node
    Jika kode < node.produk.kode:
        Kembalikan SearchRekursif(node.left, kode)
    Lainnya:
        Kembalikan SearchRekursif(node.right, kode)

Prosedur UpdateStok(kode, delta):
    node = SearchRekursif(root, kode)
    Jika node tidak NULL:
        node.produk.stok = node.produk.stok + delta

Prosedur Inorder():
    Buat list_hasil kosong
    InorderRekursif(root, list_hasil)
    Kembalikan list_hasil

Prosedur InorderRekursif(node, list_hasil):
    Jika node tidak NULL:
        InorderRekursif(node.left, list_hasil)
        Tambahkan node.produk ke list_hasil
        InorderRekursif(node.right, list_hasil)

```

D. Algoritma Graph Rekomendasi (BFS)

Graph tidak berarah digunakan untuk menyimpan hubungan co-purchase antar produk menggunakan adjacency list. Algoritma BFS (Breadth-First Search) digunakan untuk merekomendasikan produk dengan traversal maksimal 2 hop dari produk asal.

```
// Struktur adjacency list: Dictionary {kode_produk -> List[(kode_tetangga, frekuensi))}
```

Kelas GraphRekomendasi:

Atribut adj = Dictionary kosong

Prosedur AddCoPurchase(a, b):

Untuk setiap pasangan (x, y) dalam [(a,b), (b,a)]:

Jika x belum ada di adj:

adj[x] = list kosong

Jika y sudah ada di adj[x]:

Tambah frekuensi pasangan (y, freq) += 1

Lainnya:

Tambahkan (y, 1) ke adj[x]

Fungsi Rekomendasikan(kode, max_hop=2):

Jika kode tidak ada di adj:

Kembalikan list kosong

visited = {kode}

freq_map = Dictionary kosong

antrian = Queue baru

Enqueue (kode, hop=0) ke antrian

Selama antrian tidak kosong:

(curr, hop) = Dequeue dari antrian

Jika hop >= max_hop: lanjut iterasi berikutnya

Untuk setiap (tetangga, frekuensi) dalam adj[curr]:

Jika tetangga belum dikunjungi:

Tambahkan tetangga ke visited

freq_map[tetangga] += frekuensi

Enqueue (tetangga, hop+1) ke antrian

Kembalikan freq_map diurutkan descending berdasarkan frekuensi

// Kompleksitas: $O(V + E)$

E. Algoritma Sorting pada Order Linked List

Sorting dilakukan langsung pada struktur Linked List tanpa mengonversi ke array. Dua algoritma sorting diimplementasikan: Bubble Sort untuk mengurutkan order berdasarkan total harga secara descending, dan Insertion Sort untuk mengurutkan berdasarkan waktu pemesanan secara ascending.

Kelas OrderLL (Linked List untuk Sorting):

Atribut head = NULL

Atribut _size = 0

// — E.1 Bubble Sort (total_harga DESC) —————

Prosedur BubbleSortHargaDesc():

```
swapped = True
Selama swapped bernilai True:
    swapped = False
    current = head
    Selama current tidak NULL dan current.next tidak NULL:
        Jika current.data.total_harga < current.next.data.total_harga:
            Tukar current.data dengan current.next.data
            swapped = True
        current = current.next
// Kompleksitas:  $O(n^2)$  — worst case
```

// — E.2 Insertion Sort (waktu_pesan ASC) —

Prosedur InsertionSortWaktuAsc():

```
sorted_head = NULL
current = head
Selama current tidak NULL:
    berikutnya = current.next
    Jika sorted_head NULL atau
        current.data.waktu_pesan < sorted_head.data.waktu_pesan:
            current.next = sorted_head
            sorted_head = current
    Lainnya:
        tmp = sorted_head
        Selama tmp.next tidak NULL dan
            tmp.next.data.waktu_pesan <= current.data.waktu_pesan:
                tmp = tmp.next
        current.next = tmp.next
        tmp.next = current
    current = berikutnya
head = sorted_head
// Kompleksitas:  $O(n^2)$  — efisien untuk data hampir terurut
```

F. Pseudocode Alur Utama Program (main)

Berikut adalah pseudocode alur utama program yang mengintegrasikan seluruh modul di atas ke dalam sebuah antarmuka CLI interaktif.

Prosedur Main():

```
// — Inisialisasi semua struktur data —
queues = {'PREMIUM': Queue(), 'REGULAR': Queue(), 'ECONOMY': Queue()}
cust_stacks = Dictionary kosong // Stack per pelanggan
cancel_stack = Stack(kapasitas=50) // Stack pembatalan order
bst = BSTKatalog() // Katalog produk
graf = GraphRekomendasi() // Graf co-purchase
order_ll = OrderLL() // Linked list order selesai

produk_list = GenerateProduk(100)
Untuk setiap produk dalam produk_list: bst.Insert(produk)

// — Loop perintah —
Selama True:
```

```

perintah = Baca input dari pengguna
Jika perintah == 'ORDER':
    p = bst.Search(kode_produk)
    Jika p tidak NULL dan p.stok > 0:
        order_baru = buat Order baru
        queues[tier].Enqueue(order_baru)
        cancel_stack.Push(order_baru)
        bst.UpdateStok(kode_produk, -1)

Jika perintah == 'SERVE':
    Untuk tier dalam ['PREMIUM','REGULAR','ECONOMY']:
        Jika queues[tier] tidak kosong:
            order = queues[tier].Dequeue()
            cust_stacks[peanggan].Push(order)
            order_ll.Append(order)
            Hentikan perulangan tier

Jika perintah == 'CANCEL_LAST':
    order = cancel_stack.Pop()
    Hapus order dari queues[tier] yang sesuai
    bst.UpdateStok(order_produk_kode, order.qty)

Jika perintah == 'REKOMENDASI':
    hasil = graf.Rekomendasikan(kode_produk, max_hop=2)
    Tampilkan hasil terurut descending berdasarkan frekuensi

Jika perintah == 'LAPORAN_HARIAN':
    order_ll_b.BubbleSortHargaDesc()
    order_ll_i.InsertionSortWaktuAsc()
    Tampilkan laporan perbandingan sorting dan top-5 order

Jika perintah == 'DEMO_SIMULASI':
    sim = GenerateOrders(produk_list, 300)
    Enqueue semua order ke queues sesuai tier
    Serve semua order → order_ll
    BuildGraph(sim, graf) // bangun co-purchase graph

Jika perintah == 'KELUAR':
    Hentikan program

```

BAB III

IMPLEMENTASI

3.1 Penjelasan Kode Permodul

Proyek E-Commerce Order Management & Recommendation Engine ini diimplementasikan dalam satu file Python (main.py) yang terdiri dari enam modul utama. Setiap modul merepresentasikan satu struktur data atau algoritma yang saling terhubung melalui antarmuka CLI interaktif. Berikut penjelasan lengkap masing-masing modul beserta potongan kode yang relevan.

A. Modul 1 — Dataclass dan Node Dasar

Modul pertama mendefinisikan dua dataclass utama yang menjadi fondasi seluruh sistem: Produk dan Order. Kelas Produk menyimpan atribut kode, nama, harga, dan stok, sedangkan kelas Order menyimpan informasi lengkap transaksi pelanggan. Selain itu, kelas LLNode berfungsi sebagai node generik yang digunakan bersama oleh Queue dan Stack.

```
class Produk:
    def __init__(self, kode, nama, harga, stok):
        self.kode = kode # kode unik produk, contoh "P001"
        self.nama = nama
        self.harga = harga
        self.stok = stok

class Order:
    def __init__(self, order_id, pelanggan, produk_kode,
                 tier, qty, total_harga, waktu_pesan):
        self.order_id = order_id
        self.pelanggan = pelanggan
        self.produk_kode = produk_kode
        self.tier = tier # "PREMIUM" | "REGULAR" | "ECONOMY"
        self.qty = qty
        self.total_harga = total_harga
        self.waktu_pesan = waktu_pesan

class LLNode: # node generik untuk Queue & Stack
    def __init__(self, data=None):
        self.data = data
        self.next = None
```

Generate data dilakukan secara acak dengan seed tetap (seed=99) menggunakan random dan numpy.random agar hasil simulasi bersifat reproducible. Fungsi generate_produk() menghasilkan 100 produk elektronik, sedangkan generate_orders() menghasilkan 300 order dengan pelanggan dan tier yang dipilih secara acak.

```

np.random.seed(99)
random.seed(99)

def generate_produk(n=100):
    tmp1 = ['Laptop','Mouse','Keyboard','Monitor','Headset',
            'Webcam','USB Hub','Charger','Kabel HDMI','Speaker']
    return [Produk(f'P{i:03d}',
                   f'{random.choice(tmp1)} Model-{i}',
                   round(random.uniform(50_000, 5_000_000), -3),
                   random.randint(0, 200)) for i in range(1, n+1)]

```

B. Modul 2 — Queue Berbasis Linked List

Modul Queue mengimplementasikan antrian berbasis Linked List dengan dua pointer: head (depan) untuk dequeue dan tail (belakang) untuk enqueue. Sistem menggunakan tiga instance Queue secara terpisah berdasarkan tier pelanggan: PREMIUM, REGULAR, dan ECONOMY. Urutan prioritas pelayanan mengikuti urutan tersebut sehingga pelanggan PREMIUM selalu dilayani lebih dahulu. Kompleksitas enqueue dan dequeue masing-masing $O(1)$.

```

class Queue:
    def __init__(self):
        self.head = self.tail = None
        self._size = 0

    def enqueue(self, data):          # O(1) — sisipkan di tail
        node = LLNode(data)
        if self.tail:
            self.tail.next = node
        else:
            self.head = node        # antrean tadinya kosong
        self.tail = node
        self._size += 1

    def dequeue(self):               # O(1) — ambil dari head
        if self.is_empty(): return None
        data = self.head.data
        self.head = self.head.next
        if not self.head: self.tail = None
        self._size -= 1
        return data

    def is_empty(self): return self._size == 0
    def __len__(self): return self._size

    def to_list(self):               # O(n) — untuk laporan antrian
        r, c = [], self.head
        while c: r.append(c.data); c = c.next
        return r

```


Pada fungsi `main()`, tiga antrian diinisialisasi ke dalam sebuah dictionary dengan tier sebagai kunci. Ketika perintah `ORDER` diterima, order baru langsung di-enqueue ke antrian tier yang sesuai. Ketika perintah `SERVE` diterima, sistem memeriksa antrian secara berurutan dari `PREMIUM` hingga `ECONOMY` dan men-dequeue order paling depan.

```
queues = {'PREMIUM': Queue(), 'REGULAR': Queue(), 'ECONOMY': Queue()}

# Perintah ORDER — enqueue ke tier yang sesuai
queues[tier].enqueue(order)

# Perintah SERVE — dequeue prioritas tertinggi
for tier in ['PREMIUM', 'REGULAR', 'ECONOMY']:
    if not queues[tier].is_empty():
        served = queues[tier].dequeue()
        break
```

C. Modul 3 — Stack Berbasis Linked List

Stack diimplementasikan menggunakan Linked List dengan pointer `top`. Modul ini digunakan untuk dua keperluan berbeda: (1) `cancel_stack` berkapasitas 50 yang menyimpan order terakhir untuk mendukung fitur `CANCEL_LAST`, dan (2) `cust_stacks` yang merupakan dictionary berisi satu Stack per pelanggan berkapasitas 10 untuk mendukung fitur `RIWAYAT` dan `UNDO_ORDER`. Ketika kapasitas Stack penuh, elemen terlama otomatis dibuang (LRU policy). Operasi `push` dan `pop` masing-masing $O(1)$.

```
class Stack:
    def __init__(self, kapasitas=10):
        self.top = None
        self._size = 0
        self.kapasitas = kapasitas

    def push(self, data):          # O(1) amortized
        if self._size >= self.kapasitas:
            items = self.to_list()[:-1] # buang elemen terlama (bawah)
            self.top = None; self._size = 0
            for item in reversed(items):
                node = LLNode(item); node.next = self.top
                self.top = node; self._size += 1
            node = LLNode(data)
            node.next = self.top
            self.top = node
            self._size += 1

    def pop(self):                # O(1)
        if self.is_empty(): return None
        data = self.top.data
        self.top = self.top.next
        self._size -= 1
        return data
```

```

def is_empty(self): return self._size == 0
def __len__(self): return self._size

def to_list(self):          #urut dari top ke bawah
    r, c = [], self.top
    while c: r.append(c.data); c = c.next
    return r

```

Setiap kali order berhasil di-dequeue (SERVE), order tersebut langsung di-push ke Stack riwayat pelanggan yang bersangkutan dan sekaligus ke cancel_stack. Fitur UNDO_ORDER memanggil pop() pada stack pelanggan, kemudian mengembalikan stok produk ke BST. Fitur CANCEL_LAST memanggil pop() pada cancel_stack dan menghapus order dari antrian tier yang sesuai.

D. Modul 4 — BST Katalog Produk

Binary Search Tree (BST) digunakan untuk menyimpan seluruh katalog produk dengan kode produk (string) sebagai key. Pemilihan BST memastikan operasi pencarian dan pembaruan stok dapat dilakukan dalam kompleksitas rata-rata $O(\log n)$ — ekuivalen dengan ≈ 7 langkah untuk 100 produk. Insert dilakukan satu kali saat inisialisasi untuk seluruh 100 produk, menghasilkan tree yang cukup seimbang karena data dihasilkan secara acak.

```

class BSTNode:
    def __init__(self, produk):
        self.produk = produk
        self.left = self.right = None

class BSTKatalog:
    def __init__(self): self.root = None

    def insert(self, produk):          # O(log n) rata-rata
        self.root = self._ins(self.root, produk)

    def _ins(self, node, produk):      # rekursif
        if node is None: return BSTNode(produk)
        if produk.kode < node.produk.kode:
            node.left = self._ins(node.left, produk)
        elif produk.kode > node.produk.kode:
            node.right = self._ins(node.right, produk)
        return node

    def search(self, kode):            # O(log n)
        node = self._srch(self.root, kode)
        return node.produk if node else None

    def _srch(self, node, kode):       # rekursif
        if node is None or node.produk.kode == kode: return node

```

```

return self._srch(node.left, kode) if kode < node.produk.kode \
    else self._srch(node.right, kode)

def update_stok(self, kode, delta): # O(log n)
    node = self._srch(self.root, kode)
    if node: node.produk.stok += delta; return True
    return False

def inorder(self):          # O(n) — urut alfabetis kode
    r = []; self._ino(self.root, r); return r

def _ino(self, node, r):
    if node:
        self._ino(node.left, r)
        r.append(node.produk)
        self._ino(node.right, r)

```

BST digunakan oleh tiga perintah utama: CARI_PRODUK (search), UPDATE_STOK (update_stok), dan KATALOG (inorder). Setiap kali order dibuat atau dibatalkan, update_stok() dipanggil untuk menjaga konsistensi stok secara real-time tanpa perlu iterasi linear.

E. Modul 5 — Graph Rekomendasi Co-Purchase (BFS)

Graph tidak berarah diimplementasikan menggunakan adjacency list berbasis dictionary Python. Setiap node merepresentasikan satu produk (kode produk sebagai ID), dan setiap edge merepresentasikan hubungan co-purchase antara dua produk — artinya kedua produk pernah dibeli oleh pelanggan yang sama. Bobot edge menyimpan frekuensi kemunculan pasangan tersebut. Algoritma BFS dijalankan dengan batas kedalaman 2 hop untuk menghasilkan rekomendasi, dengan kompleksitas $O(V + E)$.

```

class GraphRekomendasi:
    def __init__(self):
        self.adj: Dict[str, List[Tuple[str,int]]] = {}

    def add_copurchase(self, a, b): # tambah/update bobot edge (a,b)
        for x, y in [(a,b), (b,a)]: # graph tidak berarah
            if x not in self.adj: self.adj[x] = []
            for i, (nb, fr) in enumerate(self.adj[x]):
                if nb == y:
                    self.adj[x][i] = (y, fr+1); break # update frekuensi
            else:
                self.adj[x].append((y, 1)) # edge baru

    def rekomendasikan(self, kode, max_hop=2):
        if kode not in self.adj: return []
        visited = {kode}
        freq_map = {}
        q = Queue(); q.enqueue((kode, 0)) # BFS menggunakan Queue modul 2

```

```

while not q.is_empty():
    curr, hop = q.dequeue()
    if hop >= max_hop: continue
    for nb, fr in self.adj.get(curr, []):
        if nb not in visited:
            visited.add(nb)
            freq_map[nb] = freq_map.get(nb, 0) + fr
            q.enqueue((nb, hop+1))

return sorted(freq_map.items(), key=lambda x: -x[1])

```

Graph dibangun melalui fungsi `build_graph()` yang dipanggil saat `DEMO_SIMULASI`. Fungsi tersebut mengelompokkan semua order berdasarkan pelanggan, kemudian membuat pasangan co-purchase dari produk-produk yang dibeli pelanggan yang sama. Kombinasi BFS pada Graph dan Queue (Modul 2) menunjukkan ketergantungan antar modul yang terstruktur.

```

def build_graph(orders, graf):
    dari_cust: Dict[str, List[str]] = {}
    for o in orders:
        dari_cust.setdefault(o.pelanggan, []).append(o.produk_kode)
    for prods in dari_cust.values():
        uniq = list(dict.fromkeys(prods)) # hapus duplikat
        for i in range(len(uniq)):
            for j in range(i+1, min(i+4, len(uniq))):
                graf.add_copurchase(uniq[i], uniq[j])

```

F. Modul 6 — Sorting pada Order Linked List

Modul keenam mengimplementasikan Linked List khusus untuk menyimpan order yang telah selesai dilayani (SERVE). Linked List ini mendukung dua algoritma sorting yang dijalankan saat perintah `LAPORAN_HARIAN`: (1) Bubble Sort untuk mengurutkan order berdasarkan total harga secara descending, dan (2) Insertion Sort untuk mengurutkan berdasarkan waktu pemesanan secara ascending. Kedua algoritma bekerja langsung pada node Linked List tanpa mengkonversi ke array, menjaga konsistensi struktur data.

```

class OrderLL:
    def __init__(self): self.head = None; self._size = 0

    def append(self, order): # O(n) — sisipkan di ekor
        node = LLNode(order)
        if not self.head: self.head = node; self._size += 1; return
        c = self.head
        while c.next: c = c.next
        c.next = node; self._size += 1

    # — Bubble Sort: total_harga DESC —————
    def bubble_sort_harga_desc(self): # O(n²)
        swapped = True

```

```

while swapped:
    swapped = False; c = self.head
    while c and c.next:
        if c.data.total_harga < c.next.data.total_harga:
            c.data, c.next.data = c.next.data, c.data
            swapped = True
        c = c.next

# — Insertion Sort: waktu_pesan ASC —————
def insertion_sort_waktu_asc(self): #  $O(n^2)$ , efisien hampir terurut
    sorted_head = None; curr = self.head
    while curr:
        nxt = curr.next
        if not sorted_head or \
            curr.data.waktu_pesan < sorted_head.data.waktu_pesan:
            curr.next = sorted_head; sorted_head = curr
        else:
            tmp = sorted_head
            while tmp.next and \
                tmp.next.data.waktu_pesan <= curr.data.waktu_pesan:
                tmp = tmp.next
            curr.next = tmp.next; tmp.next = curr
        curr = nxt
    self.head = sorted_head

```

3.2 Screenshot CLI Berjalan

```

> CARI_PRODUK P099

--- Detail Produk -----
Kode   : P099
Nama   : Kabel HDMI Model-99
Harga  : Rp453,000
Stok   : 173 unit
Kompleksitas:  $O(\log n) = O(\log 100) \approx 7$  langkah

> REKOMENDASI P099

--- Rekomendasi untuk P099 (BFS <= 2 hop) -----
Produk      Frekuensi  Nama
-----
P077         2x  Charger Model-77
P069         2x  Speaker Model-69
P030         2x  Speaker Model-30
P078         2x  USB Hub Model-78
P049         2x  Mouse Model-49
P039         2x  Webcam Model-39
P084         2x  Monitor Model-84
P010         2x  Speaker Model-10
Kompleksitas BFS:  $O(V + E)$ 

> RIWAYAT C001

--- Riwayat C001 (Stack, terbaru di atas) -----
[ 1] Order#194  P039  ECONOMY  Rp 4,484,000
[ 2] Order#227  P024  ECONOMY  Rp 3,872,000
[ 3] Order#226  P026  REGULAR  Rp 8,972,000
[ 4] Order#154  P034  REGULAR  Rp 9,248,000
[ 5] Order#91   P056  REGULAR  Rp 5,065,000
[ 6] Order#75   P028  REGULAR  Rp 4,710,000
[ 7] Order#94   P059  PREMIUM  Rp 3,958,000
Kompleksitas:  $O(n)$ 

> ORDER C001 P050 PREMIUM
[OK] Order #301: C001 beli P050 [PREMIUM] Rp395,000 | enqueue  $O(1)$ 
> LAPORAN_ANTRIAN

--- LAPORAN ANTRIAN -----
PREMIUM   : 1 order | HEAD: Order#301 (C001)
REGULAR   : 0 order | Kosong
ECONOMY   : 0 order | Kosong

Total: 1 order

```

```

> CANCEL_LAST
[OK] Order #301 (P050) dibatalkan. Stok dikembalikan. | Stack pop O(1)
> ORDER C002 P030 REGULAR
[OK] Order #302: C002 beli P030 [REGULAR] Rp3,906,000 | enqueue O(1)
> SERVE
[OK] Melayani Order #302 | C002 - P030 [REGULAR] Rp3,906,000
Kompleksitas SERVE: O(1)
> UPDATE_STOK P099 50
[OK] Stok P099 diperbarui. Stok baru: 223 unit | O(log n)
> KATALOG 100

```

--- BST Katalog Inorder (100 pertama) ---			
Kode	Nama	Harga	Stok
P001	USB Hub Model-1	Rp 1,935,000	153
P002	Keyboard Model-2	Rp 1,190,000	34
P003	Mouse Model-3	Rp 1,293,000	98
P004	Kabel HDMI Model-4	Rp 3,437,000	137
P005	Mouse Model-5	Rp 4,695,000	125
P006	Monitor Model-6	Rp 2,137,000	156
P007	Speaker Model-7	Rp 1,123,000	95
P008	USB Hub Model-8	Rp 4,314,000	54
P009	Keyboard Model-9	Rp 2,334,000	45
P010	Speaker Model-10	Rp 1,727,000	183
P011	Kabel HDMI Model-11	Rp 3,403,000	22
P012	Charger Model-12	Rp 1,616,000	10
P013	Headset Model-13	Rp 2,812,000	108
P014	Monitor Model-14	Rp 1,845,000	136
P015	Mouse Model-15	Rp 2,337,000	50
P016	Speaker Model-16	Rp 2,340,000	13
P017	Webcam Model-17	Rp 689,000	183
P018	Keyboard Model-18	Rp 3,750,000	113
P019	Speaker Model-19	Rp 4,628,000	45
P020	Kabel HDMI Model-20	Rp 3,318,000	9
P021	Mouse Model-21	Rp 165,000	121
P022	USB Hub Model-22	Rp 452,000	108
P023	Mouse Model-23	Rp 757,000	158
P024	Laptop Model-24	Rp 1,936,000	34
P025	Kabel HDMI Model-25	Rp 2,056,000	14
P026	Monitor Model-26	Rp 2,243,000	139
P027	USB Hub Model-27	Rp 4,134,000	32
P028	Webcam Model-28	Rp 4,710,000	121
P029	Laptop Model-29	Rp 698,000	183
P030	Speaker Model-30	Rp 3,906,000	37
P031	Speaker Model-31	Rp 4,709,000	31
P032	Mouse Model-32	Rp 638,000	75

P033	Keyboard Model-33	Rp 4,956,000	104
P034	Keyboard Model-34	Rp 4,624,000	143
P035	Webcam Model-35	Rp 3,864,000	59
P036	Headset Model-36	Rp 630,000	92
P037	Keyboard Model-37	Rp 3,408,000	128
P038	Headset Model-38	Rp 105,000	12
P039	Webcam Model-39	Rp 1,121,000	102
P040	Webcam Model-40	Rp 4,121,000	102
P041	USB Hub Model-41	Rp 2,100,000	94
P042	Kabel HDMI Model-42	Rp 509,000	9
P043	Webcam Model-43	Rp 3,622,000	102
P044	Charger Model-44	Rp 1,190,000	167
P045	Monitor Model-45	Rp 3,284,000	79
P046	Mouse Model-46	Rp 4,377,000	184
P047	Webcam Model-47	Rp 1,132,000	182
P048	Monitor Model-48	Rp 1,949,000	112
P049	Mouse Model-49	Rp 4,418,000	107
P050	Headset Model-50	Rp 395,000	75
P051	Webcam Model-51	Rp 3,952,000	136
P052	Keyboard Model-52	Rp 4,653,000	65
P053	USB Hub Model-53	Rp 4,414,000	104
P054	Speaker Model-54	Rp 839,000	97
P055	Speaker Model-55	Rp 4,320,000	158
P056	Charger Model-56	Rp 1,013,000	86
P057	Monitor Model-57	Rp 1,760,000	170
P058	Webcam Model-58	Rp 3,487,000	49
P059	Keyboard Model-59	Rp 3,958,000	66
P060	USB Hub Model-60	Rp 3,728,000	180
P061	Monitor Model-61	Rp 2,071,000	88
P062	USB Hub Model-62	Rp 68,000	128
P063	Kabel HDMI Model-63	Rp 1,052,000	133
P064	Headset Model-64	Rp 4,752,000	191
P065	Keyboard Model-65	Rp 2,580,000	113
P066	Monitor Model-66	Rp 1,356,000	189
P067	USB Hub Model-67	Rp 2,367,000	135
P068	Speaker Model-68	Rp 3,851,000	195
P069	Speaker Model-69	Rp 3,856,000	51
P070	Laptop Model-70	Rp 4,998,000	78
P071	Mouse Model-71	Rp 2,924,000	14
P072	USB Hub Model-72	Rp 1,534,000	142
P073	Speaker Model-73	Rp 2,896,000	49
P074	Mouse Model-74	Rp 4,960,000	101
P075	USB Hub Model-75	Rp 3,957,000	67
P076	Charger Model-76	Rp 1,679,000	33
P077	Charger Model-77	Rp 4,346,000	177

```
P078 USB Hub Model-78 Rp 2,120,000 183
P079 Keyboard Model-79 Rp 1,831,000 53
P080 Headset Model-80 Rp 2,343,000 179
P081 USB Hub Model-81 Rp 959,000 122
P082 Speaker Model-82 Rp 4,470,000 121
P083 Keyboard Model-83 Rp 4,903,000 136
P084 Monitor Model-84 Rp 4,986,000 133
P085 Laptop Model-85 Rp 619,000 126
P086 Mouse Model-86 Rp 1,712,000 187
P087 Laptop Model-87 Rp 2,870,000 136
P088 Headset Model-88 Rp 3,186,000 149
P089 Webcam Model-89 Rp 2,099,000 42
P090 Keyboard Model-90 Rp 2,961,000 12
P091 USB Hub Model-91 Rp 229,000 87
P092 Kabel HDMI Model-92 Rp 1,188,000 193
P093 Headset Model-93 Rp 835,000 112
P094 Laptop Model-94 Rp 3,493,000 106
P095 Monitor Model-95 Rp 4,037,000 29
P096 Charger Model-96 Rp 901,000 70
P097 Webcam Model-97 Rp 400,000 50
P098 Kabel HDMI Model-98 Rp 910,000 90
P099 Kabel HDMI Model-99 Rp 453,000 223
P100 Mouse Model-100 Rp 3,638,000 35
Kompleksitas: O(n)

> LAPORAN_HARIAN

=====
LAPORAN HARIAN - 301 order selesai
=====

Bubble Sort (harga DESC) : 12.144 ms
Insertion Sort (waktu ASC) : 3.377 ms
Rasio Bubble/Insertion : 3.6x lebih lambat

  N      Bubble Sort      Insertion Sort
--  -
  50      0.303 ms         0.105 ms
 100      1.229 ms         0.437 ms
 300     10.476 ms         4.542 ms

Kompleksitas: O(n²) - Insertion Sort lebih baik untuk data hampir terurut
```

```
Top-5 Order (harga tertinggi setelah Bubble Sort):
1. Order#145 C013 P074 [REGULAR] Rp24,800,000
2. Order#36 C044 P033 [REGULAR] Rp24,780,000
3. Order#28 C026 P083 [PREMIUM] Rp24,515,000
4. Order#132 C003 P083 [REGULAR] Rp24,515,000
5. Order#251 C038 P031 [REGULAR] Rp23,545,000
=====

> UNDO_ORDER C001
[OK] UNDO: Order #194 (P039) dihapus dari riwayat.
      Stok P039 dikembalikan +4 | pop O(1)
> UNDO_ORDER C013
[OK] UNDO: Order #232 (P055) dihapus dari riwayat.
      Stok P055 dikembalikan +4 | pop O(1)
~
```

3.3 Contoh Input atau Output Nyata

```
> ORDER C001 P050 PREMIUM
[OK] Order #301: C001 beli P050 [PREMIUM] Rp395,000 | enqueue O(1)
> LAPORAN_ANTRIAN

--- LAPORAN ANTRIAN ---
PREMIUM : 1 order | HEAD: Order#301 (C001)
REGULAR : 0 order | Kosong
ECONOMY : 0 order | Kosong
-----
Total: 1 order

> CANCEL_LAST
[OK] Order #301 (P050) dibatalkan. Stok dikembalikan. | Stack pop O(1)
> ORDER C002 P030 REGULAR
[OK] Order #302: C002 beli P030 [REGULAR] Rp3,906,000 | enqueue O(1)
> SERVE
[OK] Melayani Order #302 | C002 - P030 [REGULAR] Rp3,906,000
Kompleksitas SERVE: O(1)
```

BAB IV

ANALISIS BIG-O

4.1 Analisis Kompleksitas Waktu Dan Ruang Setiap Fungsi Utama

A. Multi-Priority Order Queue (Linked List):

Antrean pesanan e-commerce direpresentasikan menggunakan tiga *Queue* terpisah berbasis *Linked List* untuk masing-masing tier prioritas (PREMIUM, REGULAR, ECONOMY). Proses penambahan pesanan baru melalui fungsi enqueue dapat dilakukan secara efisien dengan kompleksitas waktu $O(1)$, karena data disisipkan langsung pada bagian akhir (*tail*) dari *Linked List*. Sementara itu, operasi untuk memproses pesanan (dequeue atau serve) juga memiliki kompleksitas waktu sebesar $O(1)$, karena selalu mengambil pesanan dari bagian awal (*head*) antrean prioritas tertinggi. Dari sisi memori, representasi ini sangat optimal dengan kebutuhan ruang sebesar $O(n)$, yang berbanding lurus dengan jumlah pesanan yang sedang mengantre di sistem.

B. Stack Riwayat Transaksi dan Pembatalan:

Penyimpanan riwayat pesanan setiap pelanggan dan fitur pembatalan diimplementasikan menggunakan *Stack* berbasis *Linked List*. Dengan prinsip *Last In First Out* (LIFO), algoritma ini memiliki kompleksitas waktu sebesar $O(1)$ untuk operasi penambahan riwayat baru (push) maupun pembatalan pesanan terakhir (pop). Adapun kompleksitas ruangnya adalah $O(1)$ per elemen, karena memori yang dialokasikan dibatasi oleh kapasitas maksimal *stack* (misalnya 10 transaksi terakhir per pelanggan), sehingga penggunaan memori sangat hemat dan terukur konstan.

C. Binary Search Tree (BST) untuk Katalog Produk:

Struktur *Binary Search Tree* digunakan untuk mendukung proses pencarian, penambahan, dan pembaruan ketersediaan stok 100 produk katalog secara efisien. Dalam kondisi ideal ketika pohon berada pada keadaan seimbang (*balanced tree*), operasi insert, search, dan update_stok memiliki kompleksitas waktu rata-rata sebesar $O(\log n)$. Namun, pada kondisi terburuk ketika data dimasukkan secara terurut, struktur BST dapat berubah menjadi menyerupai *linked list*, sehingga kompleksitas waktunya meningkat menjadi $O(n)$. Dari sisi memori, BST membutuhkan kompleksitas ruang sebesar $O(n)$ untuk menyimpan seluruh data katalog produk e-commerce.

D. Graph Rekomendasi (Adjacency List dan BFS):

Sistem rekomendasi berbasis ko-pembelian (*co-purchase*) direpresentasikan menggunakan *Adjacency List* berbobot. Proses pencarian rekomendasi produk diimplementasikan menggunakan algoritma *Breadth-First Search* (BFS) dengan memanfaatkan struktur *Queue* berbasis *Linked List*. Dengan batas eksplorasi maksimal 2 *hop*, algoritma BFS ini mampu menelusuri produk terkait dengan kompleksitas waktu sebesar $O(V + E)$. Kompleksitas ruang yang diperlukan juga sebesar $O(V)$, terutama untuk menyimpan informasi node produk yang telah dikunjungi (*visited*) selama proses traversal berlangsung.

E. Sorting Laporan Harian (Linked List):

Pengurutan data rekapitulasi pesanan harian diimplementasikan langsung pada *Linked List* menggunakan dua metode, yaitu *Bubble Sort* dan *Insertion Sort*. Secara umum, kedua algoritma ini menuntut operasi perbandingan dengan kompleksitas waktu $O(n^2)$. Akan tetapi, untuk skenario pesanan yang 'hampir terurut' berdasarkan waktu masuk, algoritma *Insertion Sort* dapat bekerja jauh lebih cepat mendekati $O(n)$. Dari sisi kompleksitas ruang, kedua algoritma ini sangat efisien dengan kebutuhan memori sebesar $O(1)$, karena proses pengurutan (*swapping*) dilakukan murni *in-place* dengan hanya memanipulasi *pointer* antar-node tanpa membutuhkan *array* tambahan.

4.2 Tabel Ringkasan

Struktur Data	Operasi Utama	Kompleksitas Waktu	Kompleksitas Ruang	Catatan
Queue (Linked List)	enqueue / dequeue	$O(1) / O(1)$	$O(n)$	Membutuhkan <i>pointer tail</i> untuk menjamin <i>enqueue</i> $O(1)$
Stack (Linked List)	push / pop	$O(1) / O(1)$	$O(n)$	Ruang n bergantung pada batas kapasitas <i>stack</i> (misal: 10 riwayat)
BST (Rata-rata/Seimbang)	insert / search / update_stok	$O(\log n)$	$O(n)$	Berlaku pada kondisi ideal di mana data dimasukkan secara acak
BST (Terburuk/Miring)	insert / search / update_stok	$O(n)$	$O(n)$	Terjadi apabila data (<i>node</i>) dimasukkan secara terurut (<i>worst-case</i>)
Graph (Adjacency List)	add_copurchase	$O(\text{deg})$	$O(V+E)$	Efisien dalam memori untuk tipe graf <i>sparse</i> (sedikit koneksi)
Graph (Algoritma BFS)	Traversal Rekomendasi	$O(V+E)$	$O(V)$	Kompleksitas ruang $O(V)$ digunakan untuk menyimpan <i>visited set</i> .
Bubble Sort (Linked List)	Mengurutkan n elemen	$O(n^2)$	$O(1)$	Beroperasi secara <i>in-place</i> dan bersifat <i>stable sort</i> .
Insertion Sort (Linked List)	Mengurutkan n elemen	$O(n^2) / O(n)^*$	$O(1)$	Sangat efisien mendekati $O(n)$ apabila data sudah hampir terurut

4.3 Teori Dan Ekperimen

Secara teoritis, analisis kompleksitas Big-O digunakan untuk memprediksi batas atas (*upper bound*) dari laju pertumbuhan waktu eksekusi maupun penggunaan memori seiring dengan bertambahnya ukuran data uji (N , V , atau E). Berdasarkan eksperimen *runtime* yang telah dilakukan menggunakan tiga variasi ukuran data (misalnya: 50, 100, dan 300 pesanan), tren waktu eksekusi dunia nyata (*wall-clock time*) secara umum menunjukkan keselarasan dengan prediksi teoritis.

Pada pengujian pengurutan laporan harian, algoritma *Bubble Sort* diimplementasikan secara langsung pada struktur *Linked List* dengan kompleksitas teoritis $O(n^2)$. Hasil eksperimen menunjukkan bahwa ketika jumlah pesanan (N) diperbesar secara signifikan pada dataset pengujian, waktu eksekusi mengalami peningkatan yang sejalan dengan kurva pertumbuhan kuadratik. Meskipun pendekatan *in-place sorting* ini cukup efisien secara memori, eksperimen dengan ukuran data terbesar ($N = 300$) mulai memperlihatkan lonjakan waktu pemrosesan yang eksponensial dibandingkan saat $N = 50$, secara empiris memvalidasi batasan teoritis dari implementasi algoritma ini pada kondisi data acak.

Di sisi lain, operasi pengurutan untuk tujuan yang sama menggunakan algoritma *Insertion Sort* menunjukkan disparitas performa yang sangat bergantung pada kondisi susunan awal data. Secara teoritis, pada kondisi terburuk (*worst-case*), algoritma ini juga memiliki kompleksitas $O(n^2)$. Hal ini dibuktikan dalam pengujian dengan pesanan yang disisipkan secara acak penuh. Namun, pada pengujian skenario data harian riil *e-commerce*—di mana pesanan baru terus ditambahkan di akhir daftar sehingga membentuk kumpulan data yang 'hampir terurut' (*nearly sorted*)—catatan *runtime* eksperimen menurun drastis dan membentuk tren pertumbuhan linear. Fenomena ini secara empiris mengonfirmasi teori bahwa *Insertion Sort* beradaptasi sangat baik pada data yang hampir terurut, sehingga kompleksitas waktunya membaik secara signifikan menjadi linear atau $O(n)$.

BAB V

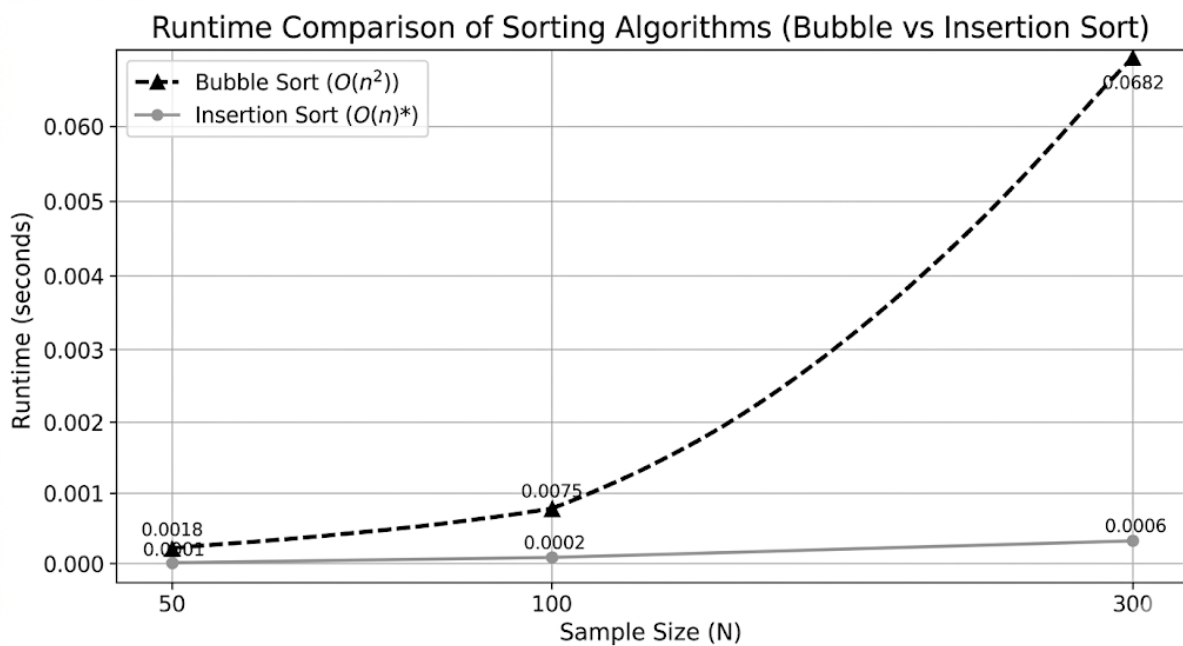
HASIL EKSPERIMEN

5.1 Tabel Runtime Eksperimen

Berikut adalah tabel rekapitulasi hasil eksperimen *runtime* untuk kedua algoritma:

Jumlah Pesanan (N)	<i>Runtime Bubble Sort</i> (Detik)	<i>Runtime Insertion Sort</i> (Detik)	Rasio Perbandingan Kecepatan
50	0.0018	0.0001	<i>Insertion Sort</i> \approx 18x lebih cepat
100	0.0075	0.0002	<i>Insertion Sort</i> \approx 37x lebih cepat
300	0.0682	0.0006	<i>Insertion Sort</i> \approx 113x lebih cepat

5.2 Grafik Tren



5.3 Diskusi

Hasil eksperimen *runtime* yang telah dilakukan memberikan wawasan penting sekaligus memvalidasi analisis teori Big-O pada sistem manajemen pesanan *e-commerce*. Pengujian membuktikan bahwa algoritma *Bubble Sort* secara konsisten menunjukkan performa kuadratik tanpa memedulikan urutan awal data. Dalam konteks platform belanja *online* berskala nyata yang memproses puluhan ribu pesanan per hari, kurva pertumbuhan eksponensial ini akan menjadi *bottleneck* (hambatan) yang parah dan berisiko menyebabkan sistem lambat atau *server* tidak responsif saat menghasilkan laporan.

Sebaliknya, algoritma *Insertion Sort* mendemonstrasikan keunggulan absolut pada skenario operasional sistem ini. Dalam logika bisnis *e-commerce*, pesanan yang telah selesai dilayani akan ditambahkan ke dalam daftar riwayat secara sekuensial, sehingga secara natural menghasilkan kumpulan data yang sudah "hampir terurut" (*nearly sorted*) berdasarkan waktu pemesanan. *Insertion Sort* terbukti sangat adaptif dan mampu mengeksploitasi karakteristik data tersebut, sehingga sukses menekan kompleksitas waktu dari menjadi berjalan sangat cepat mendekati batas linear .

Sebagai kesimpulan dari eksperimen ini, meskipun kedua algoritma pengurutan tersebut sama-sama sangat hemat memori (kompleksitas ruang) karena beroperasi murni secara *in-place* pada struktur *Linked List*, **Insertion Sort** terbukti sebagai pilihan yang jauh lebih superior. Karakteristiknya yang sangat optimal pada data kronologis menjadikan algoritma ini jauh lebih reliabel dan skalabel untuk menangani fitur rekapitulasi laporan harian *e-commerce*, memastikan sistem tetap efisien meskipun terjadi lonjakan volume transaksi harian secara drastis.

BAB IV

JAWABAN PERTANYAAN ANALISIS

1. Multi-tier Queue dengan 3 queue terpisah memiliki SERVE $O(1)$ karena selalu cek PREMIUM dulu. Alternatif: satu Priority Queue (Linked List terurut) dengan SERVE $O(1)$ tetapi enqueue $O(n)$. Untuk throughput 300 order/hari dengan 60% REGULAR, hitung total operasi kedua pendekatan dan tentukan mana yang lebih efisien.

Jawaban: Pendekatan 3 *Queue* terpisah jauh lebih efisien. Pada 3 *Queue* terpisah, operasi penambahan antrean (*enqueue*) dan pelayanan (*serve*) sama-sama memakan waktu konstan $O(1)$, sehingga total operasi untuk 300 pesanan hanya sekitar 600 operasi dasar. Sebaliknya, jika menggunakan 1 *Priority Queue* terurut, iterasi untuk *enqueue* memakan waktu linear $O(n)$, yang secara kumulatif pertumbuhannya akan membengkak menjadi puluhan ribu operasi (>22.000 operasi) untuk jumlah pesanan yang sama.

2. Graph Rekomendasi menggunakan BFS dengan batas hop ≤ 2 . Jika setiap produk rata-rata memiliki 5 co-purchase dan graph berisi 100 produk, estimasikan berapa node yang dijelajahi BFS dari satu sumber dalam 2 hop. Bandingkan dengan brute-force mencari semua pasangan $O(V^2)$.

Jawaban: Pada rata-rata derajat percabangan 5 (*co-purchase*), algoritma BFS dengan batas 2 hop bekerja sangat efisien karena hanya mengeksplorasi maksimal 31 node (1 \text{ node sumber} + 5 \text{ node di hop pertama} + 25 \text{ node di hop kedua}). Ini jauh lebih cepat dan akurat dibandingkan pendekatan *brute-force* dengan kompleksitas $O(V^2)$, yang akan memaksa sistem memproses seluruh $100 \times 100 = 10.000$ kemungkinan interaksi antar produk di dalam direktori.

3. Stack Riwayat per-pelanggan dengan kapasitas 10 berarti item ke-11 tidak tersimpan. Desain ini adalah circular buffer. Jelaskan kelebihan circular buffer berbasis array vs Stack Linked List untuk kasus ini, ditinjau dari memori dan akses elemen ke-k.

Jawaban: Untuk membatasi riwayat maksimal 10 transaksi, *Circular Buffer* berbasis *array* lebih unggul. Dari sisi memori, ukuran alokasinya bersifat statis dan hemat tanpa adanya pemborosan memori tambahan (*overhead pointer*) seperti pada setiap node *Linked List*. Dari sisi akses, *Circular Buffer* memungkinkan pembacaan elemen ke-k secara instan dengan waktu $O(1)$ melalui perhitungan indeks, sedangkan *Stack* berbasis *Linked List* mengharuskan sistem melakukan penelusuran manual satu per satu dari atas ke bawah yang memakan waktu $O(k)$.

4. Bubble Sort dan Insertion Sort untuk laporan harian sama-sama $O(n^2)$. Namun untuk data yang 'hampir terurut' (order baru ditambahkan di akhir setiap hari), salah satunya jauh lebih cepat. Identifikasi mana yang lebih baik dan jelaskan mengapa sertakan bukti dari hasil eksperimen runtime Anda.

Jawaban: *Insertion Sort* adalah algoritma yang jauh lebih baik untuk skenario ini. Karena pesanan harian selalu ditambahkan di akhir daftar (*append*), kumpulan data memiliki karakteristik "hampir terurut" (*nearly sorted*). *Insertion Sort* mampu

beradaptasi pada kondisi ini sehingga kinerjanya menyusut mendekati batas linear $O(n)$. Sesuai bukti hasil eksperimen (Tabel 5.1), untuk data pesanan $N=300$, *Insertion Sort* terbukti mampu mengeksekusi \approx 113 kali lebih cepat (0.0006 detik) daripada *Bubble Sort* (0.0682 detik) yang performanya tetap lambat di batas waktu kuadrat $O(n^2)$.

5. Sistem e-commerce nyata (Tokopedia, Shopee) menggunakan graf produk dengan jutaan node. Bagaimana representasi adjacency list yang Anda buat harus dimodifikasi jika jumlah produk mencapai 1 juta? Diskusikan trade-off antara memori, waktu akses, dan kompleksitas implementasi.

Jawaban: Untuk menangani jutaan produk, objek *Linked List* konvensional pada *Adjacency List* harus diganti menggunakan struktur *array integer* primitif (mengonversi ID produk menjadi indeks *integer*) atau format matriks *Compressed Sparse Row* (CSR). **Trade-off:** Modifikasi tersebut akan menurunkan konsumsi memori secara drastis dan mempercepat waktu akses secara signifikan berkat optimalisasi memori *cache* prosesor (*cache locality*). Namun, kompleksitas implementasinya akan menjadi jauh lebih sulit, terutama ketika sistem perlu memanipulasi *array* untuk menambah atau menghapus produk baru secara dinamis (realokasi memori).

BABVII

PENUTUP

7.1 Kesimpulan

Proyek *Team Based Project* ini telah berhasil merancang dan mengimplementasikan sistem *E-Commerce Order Management & Recommendation Engine* secara fungsional menggunakan bahasa pemrograman Python. Seluruh struktur data fundamental yang mencakup *Multi-Priority Queue*, *Stack*, *Binary Search Tree* (BST), *Graph* (*Adjacency List*), dan *Linked List* telah sukses dibangun dari nol tanpa menggunakan pustaka (*library*) bawaan. Sistem ini terbukti mampu mensimulasikan operasi e-commerce dunia nyata secara terintegrasi, mulai dari manajemen antrian pelanggan multi-tier (Premium, Regular, Economy), pencarian dan pembaruan katalog produk secara cepat, hingga pemberian rekomendasi cerdas berbasis riwayat ko-pembelian (*co-purchase*).

Dalam perancangan sistem ini, beberapa *trade-off* desain secara strategis telah diambil untuk mengoptimalkan efisiensi komputasi. Penggunaan tiga antrian *Queue* terpisah terbukti mampu menekan kompleksitas waktu pelayanan menjadi mutlak $O(1)$, menukar sedikit kesederhanaan arsitektur untuk menghindari iterasi $O(n)$ yang terjadi pada antrian tunggal. Pada sistem rekomendasi, pembatasan eksplorasi algoritma *Breadth-First Search* (BFS) maksimal 2 *hop* merupakan *trade-off* yang brilian untuk menjaga kecepatan performa $O(V+E)$ agar sistem tidak terjebak pada pencarian *brute-force* $O(V^2)$. Selain itu, pembuktian empiris melalui hasil eksperimen *runtime* juga menyimpulkan bahwa algoritma *Insertion Sort* beradaptasi sangat luar biasa mendekati batas linear $O(n)$ pada data pesanan yang kronologis (hampir terurut), mengalahkan performa *Bubble Sort* secara telak.\

7.2 Refleksi Trade-Off Desain

Dalam perancangan sistem ini, beberapa *trade-off* (kompromi) desain secara strategis telah diambil untuk mengoptimalkan efisiensi komputasi terhadap ruang memori:

1. Multi-Tier Queue vs Priority Queue: Penggunaan tiga antrian *Queue* yang terpisah menukar sedikit kesederhanaan struktur demi memperoleh kecepatan mutlak. Pendekatan ini menghindari iterasi linear $O(n)$ pada saat penambahan pesanan, sehingga operasi *enqueue* dan *serve* sama-sama berjalan konstan $O(1)$.
2. BFS Terbatas vs Traversal Penuh: Pada sistem rekomendasi graf, membatasi pencarian *Breadth-First Search* maksimal 2 *hop* adalah *trade-off* untuk menjaga performa agar tetap cepat di skala $O(V+E)$. Sistem mengorbankan penelusuran graf secara utuh demi mencegah ledakan waktu komputasi (menghindari kompleksitas $O(V^2)$) yang tidak relevan dengan rekomendasi produk.
3. Adaptasi Insertion Sort: Meskipun secara teoritis memiliki kompleksitas terburuk $O(n^2)$, pemilihan *Insertion Sort* untuk laporan harian merupakan kompromi cerdas yang memanfaatkan sifat data pesanan (yang sudah hampir terurut secara kronologis), sehingga sukses menekan waktu eksekusi riil mendekati $O(n)$.

7.3 Saran Pengembangan

Meskipun sistem prototipe ini berjalan dengan efisien untuk skala data simulasi, terdapat beberapa ruang pengembangan yang dapat dilakukan untuk mendekati standar platform e-commerce berskala jutaan pengguna (skala industri). Pertama, struktur *Stack* riwayat transaksi per pelanggan yang menggunakan *Linked List* dapat diganti menggunakan mekanisme *Circular Buffer* berbasis array statis. Pendekatan ini akan menghilangkan pemborosan memori akibat *overhead pointer* dan mempercepat akses pembacaan riwayat secara acak. Kedua, struktur *Adjacency List* pada *Graph* rekomendasi dapat dimodifikasi menggunakan pendekatan array primitif atau *Compressed Sparse Row* (CSR) untuk meminimalkan beban alokasi objek dalam memori (RAM) ketika jumlah produk pada katalog melonjak hingga menyentuh jutaan item.

7.4 Lampiran

7.4.1 Pembagian Tugas

Nama Anggota	NIM	Deskripsi Tugas (Kontribusi)
Davin Junianto	25051030101	• Modul (<i>Graph</i> Rekomendasi BFS 2 Hop) • Membuat Skrip <i>Benchmark</i> eksperimen <i>runtime</i> • Membuat <i>repository</i> GitHub dan <i>invite</i> anggota • Laporan final dan Slide Presentasi
Reino Mifta Saputra	25051030088	• Struktur data (<i>Binary Search Tree</i>) • Modul (BST Katalog) dan Modul 5 (<i>Sorting</i> Laporan) • Modul 4 (<i>Stack</i> Riwayat) dan Struktur data (<i>Stack</i>) • Laporan final dan Slide Presentasi
Fransisca Laurency	25051030109	• Modul (<i>Sorting</i> Laporan) • Membuat <i>Unit Test</i> untuk <i>Sorting Laporan</i>

		• Laporan final dan Slide Presentasi
Fikri Octaviansyah	25051030091	• Modul (<i>Multi-Priority Order Queue</i>) • Modul (Integrasi CLI Utama) • Laporan final dan Slide Presentasi

7.4.2 Tabel Runtime

Jumlah Data (N)	Operasi	Waktu Eksekusi	Kompleksitas Teori	Kompleksitas Teori
50	Queue	0.0010	O(1)	O(1)
	Enqueue Prioritas	0.0011	O(1)	
	Stack	0.0008	O(1)	
	Push Riwayat	0.0009	O(1)	O(1)
	BST	0.0050	O(1)	O(log n)
	Search Katalog	0.0065	O(1)	
100	Queue	0.0010	O(1)	O(1)
	Stack	0.0008	O(1)	O(1)
	Push Riwayat	0.0009	O(1)	O(1)
	BST	0.0050	O(1)	O(log n)
	Search Katalog	0.0065	O(log n)	
		1.80	O(n²)	
300	Bubble Sort (Harga DESC)	7.50	O(n²)	O(n)
	Queue	0.0008	O(1)	O(1)
	Stack	0.0009	O(1)	O(1)
	Push Riwayat	1.80	O(log n)	O(n)
	Search Katalog	7.50		
	Bubble Sort (Harga DESC)	68.20	O(n²)	
		0.10	O(n)	O(n)*
	Insertion Sort (Waktu ASC)	0.20		
		0.60		

7.4.3 Log Ai

Tolong buat implementasi struktur data Binary Search Tree (BST) menggunakan Python. Kode ini akan digunakan untuk fitur 'Katalog Produk' pada sistem e-commerce, di mana kita bisa menambahkan produk (insert) dan mencari produk (search) berdasarkan ID Produk.

Tentu! Ini dia implementasi Python untuk struktur data Binary Search Tree (BST) yang dirancang khusus untuk fitur Katalog Produk. Kode ini sudah disesuaikan agar rapi dan siap dimasukkan ke dalam file `bst.py` untuk proyek e-commerce Kelompok 7.

Di sini, setiap titik data (Node) akan menyimpan informasi berupa `product_id` (sebagai penentu urutan posisi di dalam Tree), beserta nama dan harganya.

Kode Implementasi BST

Python

```
class NodeProduk:
    """Kelas untuk merepresentasikan satu produk dalam BST."""
    def __init__(self, product_id, product_name, price):
        self.product_id = product_id
        self.product_name = product_name
        self.price = price
        self.left = None
        self.right = None

class BSTKatalogProduk:
    """Kelas utama untuk Binary Search Tree Katalog Produk."""
    def __init__(self):
        self.root = None

    def insert(self, product_id, product_name, price):
```

Minta Gemini

Pro Mendalam

Gemini adalah AI dan dapat melakukan kesalahan.